Advanced Algorithmic Trading Architecture for Real-Time Prediction Markets and High-Variance Assets

The rapid financialization of high-variance, time-bound events has catalyzed a paradigm shift in algorithmic trading, converging traditional quantitative finance methodologies with real-time prediction markets. Whether navigating the complexities of high-frequency cryptocurrency price movements or executing trades within the volatile, single-elimination structure of NCAA basketball tournaments—often colloquially and commercially branded under the umbrella of "Market Madness"—the underlying mathematical challenge remains identical.[1] Trading bots must continuously ingest high-dimensional, noisy data streams, execute calibrated machine learning inference engines in real-time, and route orders to Central Limit Order Books (CLOBs) while strictly managing the risk of adverse selection.[3]

Unlike traditional sportsbooks that offer static, heavily vigged odds tailored to retail bettors, modern federally regulated prediction markets operate on a continuous pricing model.[5] In these environments, contracts represent binary outcomes and fluctuate in price between $0.00 and $1.00, reflecting the real-time implied probability of an event's occurrence.[3] This microstructure mirrors traditional equities and derivatives markets, thereby enabling sophisticated market-making, statistical arbitrage, and options-based position management strategies.[7]

This comprehensive report provides an exhaustive architectural and theoretical blueprint for constructing a real-time, data-driven algorithmic trading system. The analysis dissects the requisite statistical and machine learning models for live win probability estimation, outlines the mathematical framework for embedded optionality in position management, and concludes with a rigorous Technical Specification designed for direct implementation by automated coding assistants.

Market Microstructure and the Central Limit Order Book Environment

To engineer a profitable real-time trading bot, one must first deeply understand the microstructure of the exchange it targets. Prediction markets for event contracts function via a Central Limit Order Book, where participants post resting bids and asks, providing liquidity to the market.[9] The exchange itself does not set the price; rather, price discovery is driven entirely by the continuous matching of buyer and seller sentiment, quantified through the order book's depth and spread.[6]

This dynamic creates a zero-sum environment highly susceptible to adverse selection. When a trading bot's resting limit order is filled, it explicitly means a counterparty was willing to take the opposite side of the trade at that specific price. In high-frequency environments, this often implies that the counterparty possesses superior or faster information—such as a proprietary data feed reflecting a sudden momentum shift, a player injury, or a macroeconomic shock.[4] Consequently, the trading bot cannot operate as a passive liquidity provider without implementing aggressive bluff-proofing and inertial pricing policies to guard against strategic manipulation by informed traders.[4]

The fundamental objective of the algorithm is not merely to predict the outcome of the event, but to identify moments when the CLOB's implied probability significantly diverges from the algorithm's internally computed fair value.[3] Because event contracts resolve at either maximum value or zero, holding a contract to expiration is effectively costless in terms of traditional exit fees or rolling costs.[3] This structural reality transforms every purchased contract into an American-style binary option, granting the algorithm the right—but not the obligation—to sell the position back into the market prior to settlement if the price moves favorably.[3]

Data Engineering and Feature Architecture

The efficacy of any predictive model is intrinsically bound to the quality, dimensionality, and latency of its underlying data architecture. Real-time trading models require dual ingestion pipelines: one for the underlying asset's state (e.g., live play-by-play data, tick-level cryptocurrency order flow) and one for the exchange's market state (the L2 order book).[3]

High-Dimensional Feature Spaces in Financial Time Series

In scenarios demanding high-frequency forecasting, such as predicting short-term cryptocurrency price movements, feature engineering is paramount. Analysis of proprietary trading datasets—such as those utilized in quantitative competitions involving over 525,000 minutes of financial time series data—reveals that standard market features like bid, ask, and volume quantities often exhibit exceedingly weak predictive signals.[1] In specific evaluations, raw market features demonstrated a maximum Pearson correlation of merely 0.0158 with future price movements.[1]

To extract actionable alpha, quantitative researchers rely on expansive matrices of anonymized, proprietary trading signals. In high-dimensional spaces involving nearly 900 features, the correlation with the target variable can increase significantly, achieving correlations approaching 0.0677.[1] The target variable itself is typically highly volatile, characterized by significant variance and heavy-tailed distributions.[1] The data engineering pipeline must normalize these features in real-time without introducing forward-looking bias, utilizing rolling z-scores and online variance estimation to ensure the machine learning models receive stable inputs regardless of macroeconomic regime shifts.

State Space Representation in Event Prediction

Conversely, when modeling live sports events such as NCAA basketball games, the feature space is significantly lower in dimensionality but highly sensitive to temporal decay. The state space of a live basketball game can be represented by a continuous vector capturing the immediate context of the match.[14] This vector must minimally include the current score differential, the exact time remaining in seconds, a binary indicator of current possession, and disparities in foul counts and remaining timeouts.[14]

However, relying solely on current in-game statistics ignores the inherent skill disparity between the two competing entities. A highly accurate model must anchor its live predictions to a robust pre-game prior.[14] This prior acts as a measure of relative team strength and prevents the model from violently overreacting to early-game variance, such as an underdog scoring the first three baskets.[16] The data pipeline must therefore integrate pre-game efficiency metrics, historically sourced from advanced statistical databases tracking adjusted offensive efficiency, defensive efficiency, and tempo.[17] Advanced implementations will also ingest granular metrics such as defensive rebound percentages, effective field goal rates, and proprietary ShotQuality metrics that quantify the underlying scoring efficiency independent of realized outcomes.[12]

Pre-Trade Modeling: Establishing the Statistical Prior

Before the algorithm can engage in live trading, it must generate a mathematically sound baseline probability, or prior, which serves as the foundation for all subsequent real-time updates. Relying on public consensus or tournament seedings introduces massive inefficiencies, as public markets frequently overvalue narrative-driven teams and underestimate statistically superior mid-majors.[20]

To synthesize historical performance metrics into a singular pre-game win probability, statistical models traditionally deploy multinomial logistic regression or generalized linear models.[22] By regressing season-average statistics against historical match outcomes, the model isolates the predictive weight of each feature. A standard formulation predicts the log-odds of victory based on the differential in adjusted offensive and defensive efficiencies, appropriately scaled by the anticipated pace of play.[22]

While logistic regression provides excellent interpretability and fast execution, it struggles to capture complex, non-linear interactions between opposing team styles.[16] To capture these nuances, quantitative developers increasingly utilize gradient boosting frameworks such as XGBoost, or ensemble learning methods combining random forests with stochastic gradient descent classifiers.[20] These models excel at identifying conditional probabilities—for instance, recognizing that an underdog's high variance in three-point shooting dramatically increases their probability of achieving an upset against a slow-paced favorite.[25]

For macro-level tournament forecasting and bracket optimization, models leverage Monte Carlo simulations. By running tens of thousands of simulated tournaments utilizing the baseline priors, the system maps out complex path dependencies and conditional advancement probabilities, identifying specific teams that offer significant expected value against the consensus betting market.[26] However, while Monte Carlo methods are invaluable for pre-tournament analysis, their computational overhead renders them unsuitable for millisecond-level live order book execution.

Real-Time Machine Learning Inference Engines

Once the market goes live, the algorithm's pricing engine must transition from static prior generation to continuous, real-time probability updating. The selection of the underlying machine learning architecture is the most critical determinant of the trading bot's profitability. The model must balance raw predictive power against the strict latency constraints of high-frequency execution.

High-Dimensional Regression and Neural Network Architectures

In complex financial time series prediction, such as short-term cryptocurrency forecasting, empirical data demonstrates a counterintuitive reality: overly complex deep learning architectures frequently underperform heavily regularized linear models.[1] Extensive testing on large-scale proprietary datasets indicates that a carefully tuned Ridge Regression model, supported by rigorous feature engineering, can achieve superior Pearson correlation coefficients (0.1175) compared to complex three-layer Multi-Layer Perceptrons (MLPs) paired with AutoEncoders (0.0807).[1]

The underperformance of deep neural networks in this specific context is often attributed to the extreme noise-to-signal ratio inherent in financial markets. Complex MLPs are highly prone to overfitting the training data, memorizing market noise rather than extracting generalizable alpha.[1] The Ridge Regression model, by contrast, applies an L2 regularization penalty (alpha=1.0) that forces the model to distribute its weights smoothly across all 896 features, preventing any single noisy indicator from dominating the prediction.[1] For a real-time trading bot, implementing a regularized linear model ensures robust, noise-resistant predictions that generalize well to unseen market regimes, while also offering inference times measured in microseconds.

Interval-Based State Space Modeling

When transitioning from asset price forecasting to live event probability modeling, the architecture must account for the deterministic nature of time. In a live basketball game, the relationship between score differential and win probability is highly non-linear with respect to time remaining.[16] A singular logistic regression model trained across the entire dataset performs poorly, as it cannot comprehend that a five-point lead with thirty minutes remaining

carries drastically less certainty than a five-point lead with thirty seconds remaining.[16]

To resolve this temporal non-linearity, algorithmic implementations utilize fixed-time interval modeling.[16] By segmenting the game into distinct temporal buckets, independent models are trained for each specific phase of the event. As time decays, the coefficients assigned to the score differential naturally scale upward, mathematically mirroring the increasing gravity of each point.[29] This interval-based approach provides a highly reactive and computationally lightweight inference engine capable of updating fair value instantaneously upon receipt of a new score tick.

Deep Sequence Modeling and Probability Calibration

For the most advanced implementations, state-of-the-art systems utilize sequence models such as Long Short-Term Memory (LSTM) networks or Transformer architectures.[30] Unlike interval regressions that view each game state in isolation, sequence models ingest the entire chronological history of play-by-play data, allowing the network to build a latent representation of game momentum, fatigue, and strategic shifts.[31] These deep learning models can inherently detect complex patterns, such as the probability of a blowout expanding or the likelihood of a late-game regression to the mean.[12]

However, the output of a neural network in a binary classification task is a raw sigmoid activation, which does not inherently represent a true mathematical probability. A model that outputs a 0.85 confidence score does not necessarily mean the event will occur 85% of the time.[13] In predictive trading, uncalibrated probabilities are fatal, as the algorithm relies on these exact percentages to calculate expected value against the market's bid-ask spread.[13]

To enforce strict probability calibration, the neural network must be trained utilizing the Brier Score as its primary loss function.[5] The Brier Score explicitly measures the mean squared error between the predicted probability and the actual binary outcome, heavily penalizing overconfident, incorrect predictions.[30] A model optimized via Brier Score ensures that when it communicates a 65% win probability to the execution engine, the underlying asset historically achieves victory exactly 65% of the time under identical state conditions, thereby securing mathematical alignment between the model's confidence and market reality.[5]

Algorithmic Architecture	Primary Application Domain	Latency Profile	Key Optimization Metric
Ridge Regression (L2 Regularized)	High-dimensional financial time series, crypto price	Ultra-low (Microseconds)	Pearson Correlation, Noise reduction via L2

	movements.[1]		penalty.[1]
Interval Logistic Regression	Live event state-space modeling, in-game win probability.[16]	Ultra-low (Microseconds)	Cross-Entropy, addressing temporal non-linearity.[16]
LSTM / Transformer Networks	Sequential play-by-play analysis, momentum detection.[30]	Moderate (Milliseconds)	Brier Score, strict probability calibration.[30]
XGBoost / Gradient Boosting	Pre-game prior generation, complex feature interactions.[1]	Low (Sub-millisecond)	Log-Loss, target encoding accuracy.[1]

Quantitative Options-Based Position Management

Generating an accurate, real-time probability is merely the prerequisite for algorithmic trading; the true edge is derived from sophisticated position management. Prediction markets exhibit unique microstructural behavior because contracts resolve definitively at binary endpoints, allowing participants to hold positions to expiration without incurring continuous carrying costs or final exit fees.[3] This free settlement dynamic fundamentally alters the mathematics of risk management.

Every contract acquired by the trading bot must be mathematically treated as an American-style binary option.[3] The intrinsic value of the contract at any given moment is equal to the machine learning model's calibrated win probability. However, because the market remains open and continuously traded throughout the event, the bot retains the right to sell the contract back into the liquidity pool if the market price surges favorably. This right to sell carries quantifiable embedded value.[3]

An unsophisticated algorithm would simply sell its inventory the moment the market bid exceeds its internal fair value by a single tick. This is mathematically suboptimal because it surrenders the embedded option value for a negligible premium. The algorithm must demand that the market significantly overprice the contract to justify relinquishing the optionality of holding a winning position to free settlement.[3]

To calculate this embedded value dynamically, quantitative models deploy closed-form formulas calibrated to extensive backward induction over massive state-space grids.[3] Extensive empirical modeling of live prediction markets yields a robust, three-parameter formula to

quantify this option value (OV) in real-time [3]:

$$OV = 0.39 \times N^{0.42} \times (1 + 16.12 \times p(1-p))$$

Within this formulation, p represents the algorithm's current calibrated win probability for the underlying asset. The variable N is a function of the time remaining in the event, specifically defined as $N = \frac{T_{rem}}{120}$, representing the discrete number of independent evaluation opportunities remaining before settlement.[3]

This formula elegant captures the volatility dynamics of binary event markets. The term $p(1-p)$ ensures that the option value peaks when the game is perfectly contested ($p = 0.50$) and collapses as the probability approaches definitive resolution (0.0 or 1.0).[3] Concurrently, the temporal multiplier $N^{0.42}$ dictates that optionality decays non-linearly as time expires. By continuously computing this OV metric, the execution engine dynamically tightens its exit thresholds as the event progresses, holding out for massive premiums early in the event while aggressively locking in marginal profits when time runs short and optionality evaporates.[3]

Execution Logic and Stringent Risk Mitigation

With the pricing and optionality engines actively computing fair value, the algorithm relies on a rigid, deterministic execution gateway to route orders. To protect against the severe adverse selection inherent in high-variance CLOBs, the execution logic evaluates every potential trade against strict environmental filters before calculating entry and exit thresholds.[3]

Environmental Filtering and Spread Constraints

The algorithm must defensively monitor the health of the order book and the latency of its data feeds. The primary defense against toxic order flow is the implementation of a strict freshness constraint. The system validates the timestamp of the latest game state update; if the data provider's feed has not reflected a score or clock change within a trailing 15-second window, the system enters a hard cooldown.[3] Executing trades on stale data exposes the bot to "phantom edges," where the market has already reacted to a televised event that the API has not yet broadcast, resulting in immediate algorithmic losses.[3]

Furthermore, the system strictly polices the bid-ask spread. Trading is suspended if the spread exceeds 3 cents, as wide spreads dictate illiquid conditions where transaction costs and slippage entirely negate the model's theoretical edge.[3] The algorithm also restricts trading to price bands strictly between 10 cents and 90 cents. Deep out-of-the-money or deep

in-the-money contracts carry disproportionately high percentage fee drags relative to their expected value, rendering them mathematically unviable for high-frequency scalping.[3] Finally, trading is hard-halted during the final minutes of an event, as the terminal phases of high-variance matches involve unpredictable fouling strategies and chaotic variance that break probabilistic calibration.[3]

Dynamic Threshold Computation

When all environmental filters are successfully cleared, the execution engine calculates precise entry and exit parameters.

For entry, the algorithm does not simply buy when the price falls below fair value. It demands a required edge that dynamically scales based on the time elapsed in the event, typically requiring a 5% margin of safety in the early phases and scaling up to a 15% margin as time decays and variance condenses.[3] If the market's best ask price satisfies this strict margin beneath the model's computed probability, an entry order is generated.

The exit logic relies entirely on the option-pricing framework. The algorithm evaluates its current inventory and calculates a hard exit threshold utilizing the previously derived OV and the exchange's specific fee structure.[3] The inequality governing the exit execution is defined as [3]:

$$Bid > (p \times 100) + OV + Fees$$

Consider an operational scenario where a match is tied with exactly 20 minutes remaining, and the machine learning model dictates a 60% probability of victory ($p = 0.60$). The options engine calculates an embedded value of 5.0 cents based on the volatility and time decay parameters.[3] Assuming the exchange levies a variable fee calculated at 1.7 cents, the execution threshold is explicitly set at $60 + 5.0 + 1.7 = 66.7$ cents.[3] The algorithm will stubbornly refuse to sell its inventory unless a counterparty bids 67 cents or higher, demanding a massive 7-cent overpayment from the market to surrender its optionality. If the market refuses to meet this exorbitant price, the algorithm gracefully holds the position to free settlement, completely insulated from unnecessary transaction costs.[3]

Bankroll Management via Fractional Kelly

Executing mathematically sound +EV trades is irrelevant if position sizing is fundamentally flawed. In prediction markets, where the algorithm is exposed to unquantifiable black-swan events—such as sudden catastrophic player injuries or severe data feed outages—betting overly aggressive fractions of the portfolio guarantees eventual ruin.[13] The trading architecture mandates the use of dynamic sizing protocols derived from the Kelly Criterion.[13]

The classical Kelly formula identifies the mathematically optimal fraction of the bankroll to risk to maximize compound growth. However, because prediction market models possess inherent confidence intervals and structural uncertainty [15], the system must deploy a Fractional Kelly strategy. By strictly capping the sizing multiplier to a fraction of the Kelly output (e.g., 0.15 Kelly or 0.25 Kelly), the algorithm violently suppresses portfolio variance and secures survival during inevitable periods of algorithmic drawdown, allowing the long-term mathematical edge to dictate the equity curve.[13]

Technical Specification Document for Automated Implementation

This exhaustive technical specification is architected explicitly for ingestion by a Large Language Model or automated coding assistant. It provides the definitive structural blueprint necessary to implement the real-time prediction market trading algorithm in a production environment.

1. Core System Architecture and Technology Stack

Objective: Engineer an asynchronous, ultra-low latency, event-driven algorithmic trading bot capable of ingesting live state data, running millisecond machine learning inference, and executing complex options-based limit orders on a Central Limit Order Book.[3] **Primary Language**: Python 3.11+ (Leveraging strict static typing via mypy and pydantic for data validation). **Concurrency Paradigm**: The system must avoid thread-locking by utilizing asyncio for all I/O bound operations, specifically deploying aiohttp for REST API execution and websockets for continuous data streaming. **Data Processing & Inference**: State vectors are processed via numpy, while the underlying machine learning models must be serialized using ONNX or compiled xgboost binaries to bypass the Python Global Interpreter Lock (GIL) and achieve sub-millisecond inference execution. **State Synchronization**: Inter-process communication and global state management must be handled via a localized Redis instance, ensuring that the ingestion pipelines do not block the execution gateway.

2. Decoupled Module Design

The architecture dictates strict separation of concerns, decoupled into four highly specialized asynchronous daemons communicating exclusively via non-blocking asyncio.Queue structures.

2.1 Asynchronous Data Ingestion Engine

- **CLOB WebSocket Listener**: Establishes a persistent, authenticated WebSocket connection to the prediction market exchange. It ingests delta-updates to the L2 order book and maintains a highly optimized, in-memory bid-ask replica. Instantiates and dispatches strictly typed MarketUpdateEvent payloads to the queue.
- **Game State API Poller**: Interfaces with high-frequency sports data providers (e.g., ESPN API, SportsData.io).[17] Due to the lack of WebSocket availability for many sports data

providers, this daemon implements an aggressive asynchronous polling loop. It extracts the raw telemetry: home_score, away_score, time_remaining_seconds, and possession. Dispatches GameStateEvent payloads.

- **Latency & Synchronization Monitor**: Crucially, this sub-module timestamps every incoming GameStateEvent. It maintains a global boolean flag, DATA_STALE. If the current system time exceeds the last received game state timestamp by more than 15 seconds, it forcefully sets DATA_STALE = True, instantly triggering a global trading halt across all downstream modules.[3]

2.2 Machine Learning Inference Pipeline

- **Prior Initialization**: Upon instantiation, this module executes a singular REST call to fetch pre-game efficiency databases (e.g., KenPom metrics) and historic closing lines to instantiate the base prior probability.[17]
- **Live Inference Loop**: Subscribes to the GameStateEvent queue. Upon receipt of a new event, it constructs the dense feature vector.
  - *Execution*: The vector is passed to the compiled interval-logistic regression or lightweight Ridge regression model.[1]
  - *Output*: Returns the strictly calibrated probability variable, p.[5]
- **Options Pricing Engine**: Immediately processes p through the closed-form embedded value formula to calculate the real-time OV in cents.[3]
- Dispatches a comprehensive PricingUpdateEvent containing the game ID, the computed p, and the calculated OV.

2.3 Strategy Gateway and Risk Evaluation

- This module acts as the algorithmic brain, merging the MarketUpdateEvent stream with the PricingUpdateEvent stream.
- **Environmental Firewall**: Executes sequential boolean checks. If DATA_STALE is True, or if time_remaining_seconds < 240, or if the absolute difference between best_ask and best_bid exceeds 3.0 cents, the evaluation loop immediately returns, discarding the event and preventing execution.[3]
- **Entry Matrix**:
  - Computes the dynamically scaling required_edge based on elapsed time.[3]
  - Evaluates the inequality: best_ask <= (p * 100) - required_edge.
  - Enforces price boundary constraints: 10.0 <= best_ask <= 90.0.[3]
  - If all conditions are met, it triggers a Fractional Kelly calculation to determine optimal sizing and dispatches a BuyExecutionEvent.
- **Exit Matrix**:
  - Queries the internal ledger for existing inventory on the specific asset.
  - If inventory exists, it constructs the exit threshold: exit_price = (p * 100) + OV +

current_exchange_fee.[3]
    - Evaluates the inequality: best_bid > exit_price.
    - If satisfied, dispatches a SellExecutionEvent to liquidate the position and capture the premium.

2.4 Order Routing and Execution Manager

- Subscribes exclusively to Execution Events.
- Translates the internal logical directives into exchange-specific API payloads.
- Routes orders utilizing Immediate-Or-Cancel (IOC) flags to prevent limit orders from resting in a rapidly shifting book and getting adversely filled during a momentum swing.
- Listens for exchange execution confirmations and updates the local inventory ledger and unencumbered capital balances.

Architectural Module	Primary Responsibility	Data Structures utilized	Failure Mode Handling
Ingestion Engine	Maintain localized L2 book and Game State.[3]	WebSockets, Asyncio Queues.	Reconnect with exponential backoff; trigger system-wide trading halt.
Inference Pipeline	Sub-millisecond probability and Option Value calculation.[3]	Numpy Arrays, ONNX runtime.	Fallback to pre-game prior if live feature vector is malformed.
Strategy Gateway	Evaluate EV against environmental and liquidity filters.[3]	Pydantic validation models.	Fails safe (no trade) if any spread or latency parameter is breached.
Execution Manager	Route IOC orders, manage Fractional Kelly sizing.[3]	REST API POST requests.	API rate-limit throttling via Token Bucket algorithm.

3. Failsafes and Infrastructure Resilience

Operating an autonomous financial algorithm requires extreme defensive programming. The system must implement a strict Token Bucket rate-limiting algorithm on all outgoing REST requests to ensure compliance with exchange API guidelines, preventing temporary IP

blacklisting.

A global circuit breaker must be instantiated within the Execution Manager. The circuit breaker monitors a rolling 60-second execution window. If the algorithm attempts to fire an anomalous number of orders—indicative of a logic loop or an extreme latency arbitrage scenario where the exchange is failing to process cancellations—the circuit breaker trips, instantly canceling all resting orders and gracefully shutting down the Python process, dispatching a critical alert to the administrator.

By strictly adhering to this technical blueprint, quantitative developers can deploy a highly resilient, mathematically optimal trading system capable of extracting alpha from the noise and volatility inherent in real-time prediction markets.

Works cited

1. DRW Crypto Market Prediction - Kaggle Competition Solution achieving 89.7% of winner performance with comprehensive EDA analysis and feature engineering pipeline - GitHub, accessed on March 19, 2026, https://github.com/coderback/drw-crypto-market-prediction
2. Free Entry, $2M in Prizes | Market Madness 2026 Stock Bracket Challenge Begins, accessed on March 19, 2026, https://www.youtube.com/watch?v=ayekVOzu0o4
3. isaiahnick/ncaam-live-trader: Automated live trading system for NCAA basketball prediction markets on Kalshi. Bayesian win probability model, options-based position management with quantified embedded optionality, and sub-second execution infrastructure. - GitHub, accessed on March 19, 2026, https://github.com/isaiahnick/ncaam-live-trader
4. Dynamic Learning and Market Making in Spread Betting Markets with Informed Bettors - IORA – Institute of Operations Research and Analytics, accessed on March 19, 2026, https://iora.nus.edu.sg/wp-content/uploads/2022/09/SSRN-id3283392.pdf
5. Prediction Markets Statistics 2026: Market Size, Growth & Trends - Gambling Insider, accessed on March 19, 2026, https://www.gamblinginsider.com/in-depth/110180/prediction-market-statistics
6. Social Betting: Powering the Future of Sports Prediction Markets - DataArt, accessed on March 19, 2026, https://www.dataart.com/blog/from-sports-to-social-unlocking-new-revenue-streams-with-sports-prediction-markets-by-russell-karp
7. STOCK DRAFT Live 12 EST! Market Madness 2026: The Ultimate March Tournament | Win 100K FLEX & More - YouTube, accessed on March 19, 2026, https://www.youtube.com/watch?v=djE3StoNQvQ
8. Prediction markets are dominating college athletics, but no one is talking about it, accessed on March 19, 2026, https://acuoptimist.com/2026/03/prediction-markets-are-dominating-college-athletics-but-no-one-is-talking-about-it/
9. market-maker-bot · GitHub Topics, accessed on March 19, 2026,

https://github.com/topics/market-maker-bot?o=desc&s=forks
10. kalshi · GitHub Topics, accessed on March 19, 2026, https://github.com/topics/kalshi?o=asc&s=stars
11. March Madness West Region Preview: NCAA Tournament Markets, accessed on March 19, 2026, https://www.thelines.com/prediction-markets/trending/march-madness-west-region-preview-ncaa-tournament-prediction-markets/
12. ShotQuality: Real-Time Sports Intelligence for Live Markets, accessed on March 19, 2026, https://shotquality.com/
13. How to Spot Value Bets Using Advanced Statistical Models - The Emory Wheel, accessed on March 19, 2026, https://www.emorywheel.com/article/how-to-spot-value-bets-using-advanced-statistical-models-20260222
14. A Simple Neural Network for In-Game Win Probability Modeling of NCAA Basketball Games | by Evan Zamir | Analytics Vidhya | Medium, accessed on March 19, 2026, https://medium.com/analytics-vidhya/a-simple-neural-network-for-in-game-win-probability-modeling-of-ncaa-basketball-games-58ab4e3ca0f9
15. How Win Probability Models Work in College Football - BettorEdge, accessed on March 19, 2026, https://www.bettoredge.com/post/how-win-probability-models-work-in-college-football
16. NCAA Basketball Win Probability Model | Yale Undergraduate Sports Analytics Group, accessed on March 19, 2026, https://sports.sites.yale.edu/ncaa-basketball-win-probability-model
17. hoopR • Data and Tools for Men's Basketball • hoopR, accessed on March 19, 2026, https://hoopr.sportsdataverse.org/
18. Package index - hoopR - SportsDataverse, accessed on March 19, 2026, https://hoopr.sportsdataverse.org/reference/index.html
19. Predicting the March Madness Champion Using Statistical Modeling - Bryant Digital Repository, accessed on March 19, 2026, https://digitalcommons.bryant.edu/cgi/viewcontent.cgi?article=1010&context=honors_cis
20. March Madness Prediction Using Machine Learning Techniques, accessed on March 19, 2026, https://run.unl.pt/bitstream/10362/33864/1/TGI0135.pdf
21. Most probable first-round upsets in the men's 2026 NCAA tournament, accessed on March 19, 2026, https://www.tsn.ca/ncaa/article/most-probable-first-round-upsets-in-the-mens-2026-ncaa-tournament-n1-48223542/
22. We Built a Live Win Probability Engine for Our March Madness Survivor Pool - Reddit, accessed on March 19, 2026, https://www.reddit.com/r/sportsanalytics/comments/1ruiu08/we_built_a_live_win_probability_engine_for_our/
23. The ultimate guide to predictive college basketball analytics - THE POWER RANK, accessed on March 19, 2026, https://thepowerrank.com/cbb-analytics/

24. How to Use Machine Learning to Predict NCAA March Madness - Analytics8, accessed on March 19, 2026, https://www.analytics8.com/blog/how-to-use-machine-learning-to-predict-ncaa-march-madness/
25. Comparing Classical and Modern Models for Predicting NCAA Basketball Win Percentage, accessed on March 19, 2026, https://rpubs.com/markstanley/1385156
26. The Best Statistical Models For Sports Betting - Sports Prediction Asia, accessed on March 19, 2026, https://www.sportsprediction.asia/blog-detail/348/the-best-statistical-models-for-sports-betting.html
27. March Madness bracket predictions using Monte Carlo - Lumivero, accessed on March 19, 2026, https://lumivero.com/resources/blog/march-madness-predictions-with-risk/
28. NCAA Tournament Monte Carlo Simulation : r/CollegeBasketball - Reddit, accessed on March 19, 2026, https://www.reddit.com/r/CollegeBasketball/comments/5pn57h/ncaa_tournament_monte_carlo_simulation/
29. basic-nba-tutorials/win_probability/make_win_probability_model.md at main - GitHub, accessed on March 19, 2026, https://github.com/anpatton/basic-nba-tutorials/blob/main/win_probability/make_win_probability_model.md
30. Forecasting NCAA Basketball Outcomes with Deep Learning: A Comparative Study of LSTM and Transformer Models, accessed on March 19, 2026, https://arxiv.org/html/2508.02725v1
31. Trader's View | Think all basketball is the same? Think again. | Post - Genius Sports, accessed on March 19, 2026, https://www.geniussports.com/content-hub/traders-view-think-all-basketball-is-the-same-think-again/
32. Machine Learning Sports Predictions Behind Big Wins - WSC Sports, accessed on March 19, 2026, https://wsc-sports.com/blog/industry-insights/machine-learning-sports-predictions-behind-big-wins/
33. Social Prediction Markets: The Next Evolution in Sports Betting - BettorEdge, accessed on March 19, 2026, https://www.bettoredge.com/post/social-prediction-markets-the-next-evolution-in-sports-betting
34. NCAA College Basketball Data API Developer Portal - SportsDataIO, accessed on March 19, 2026, https://sportsdata.io/developers/api-documentation/ncaa-basketball